

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Обработка изображений в ИС»

Тема: «Конструирование моделей на базе предобученных нейронных сетей»

Выполнил:
Студент 4 курса
Группы ИИ-26
Шоева Е.Т.
Проверила:
Андренко К.В.

Цель: осуществлять обучение НС, сконструированных на базе предобученных архитектур НС

Ход работы

Общее задание

1. Для заданной выборки и архитектуры предобученной нейронной организовать процесс обучения НС, предварительно изменив структуру слоев, в соответствии с предложенной выборкой. Использовать тот же оптимизатор, что и в ЛР №1. Построить график изменения ошибки и оценить эффективность обучения на тестовой выборке;
2. Сравнить полученные результаты с результатами, полученными на кастомных архитектурах из ЛР №1;
3. Ознакомиться с state-of-the-art результатами для предлагаемых выборок (по материалам в сети Интернет). Сделать выводы о результатах обучения НС из п. 1 и 2;
4. Реализовать визуализацию работы предобученной СНС и кастомной (из ЛР 1). Визуализация осуществляется посредством выбора и подачи на сеть произвольного изображения (например, из сети Интернет) с отображением результата классификации;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

В-т	Выборка	Оптимизатор	Предобученная архитектура
11	MNIST	Adadelata	ResNet34

```
import os
```

```
import random
```

```
import numpy as np
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
import torch.optim as optim
```

```
from PIL import Image
```

```
from torch.utils.data import DataLoader
```

```
from torchvision import datasets, transforms
```

```
from torchvision.models import ResNet34_Weights, resnet34
```

```
import matplotlib.pyplot as plt
```

```
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
RANDOM_SEED = 42
```

```
torch.manual_seed(RANDOM_SEED)
```

```
random.seed(RANDOM_SEED)
```

```
np.random.seed(RANDOM_SEED)
```

```
MNIST_MEAN, MNIST_STD = (0.1307,), (0.3081,)
```

```
IMAGENET_MEAN, IMAGENET_STD = (0.485, 0.456, 0.406), (0.229, 0.224, 0.225)
```

```
CUSTOM_CNN_EPOCHS = 5
```

```
CUSTOM_CNN_BATCH = 64
```

```
RESNET_IMAGE_SIZE = 64
```

```
RESNET_EPOCHS = 3
```

```
RESNET_BATCH = 128
```

TEST_BATCH = 1000

```
cnn_transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize(MNIST_MEAN, MNIST_STD),  
])
```

```
resnet_transform = transforms.Compose([  
    transforms.Grayscale(num_output_channels=3),  
    transforms.Resize((RESNET_IMAGE_SIZE, RESNET_IMAGE_SIZE)),  
    transforms.ToTensor(),  
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),  
])
```

```
cnn_train_dataset = datasets.MNIST(root="./data", train=True, download=True,  
transform=cnn_transform)  
cnn_test_dataset = datasets.MNIST(root="./data", train=False, download=True,  
transform=cnn_transform)
```

```
resnet_train_dataset = datasets.MNIST(root="./data", train=True, download=True,  
transform=resnet_transform)  
resnet_test_dataset = datasets.MNIST(root="./data", train=False, download=True,  
transform=resnet_transform)
```

```
cnn_train_loader = DataLoader(cnn_train_dataset, batch_size=CUSTOM_CNN_BATCH,  
shuffle=True)  
cnn_test_loader = DataLoader(cnn_test_dataset, batch_size=TEST_BATCH, shuffle=False)
```

```
resnet_train_loader = DataLoader(resnet_train_dataset, batch_size=RESNET_BATCH,  
shuffle=True)  
resnet_test_loader = DataLoader(resnet_test_dataset, batch_size=TEST_BATCH, shuffle=False)
```

```
class SimpleCNN(nn.Module):  
    def __init__(self, num_classes: int = 10):  
        super().__init__()  
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)  
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)  
        self.pool = nn.MaxPool2d(2, 2)  
        self.fc1 = nn.Linear(32 * 7 * 7, 128)  
        self.fc2 = nn.Linear(128, num_classes)  
        self.dropout = nn.Dropout(0.25)  
  
    def forward(self, x):  
        x = self.pool(F.relu(self.conv1(x)))  
        x = self.pool(F.relu(self.conv2(x)))  
        x = x.view(x.size(0), -1)  
        x = F.relu(self.fc1(x))  
        x = self.dropout(x)  
        return self.fc2(x)
```

```

def build_pretrained_resnet34(num_classes: int = 10):
    weights = ResNet34_Weights.DEFAULT
    model = resnet34(weights=weights)

    for param in model.parameters():
        param.requires_grad = False

    for param in model.layer4.parameters():
        param.requires_grad = True

    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

```

```

def train_one_epoch(model, loader, optimizer, criterion):
    model.train()
    running_loss = 0.0
    for data, target in loader:
        data, target = data.to(DEVICE), target.to(DEVICE)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    return running_loss / len(loader)

```

```

def evaluate(model, loader, criterion):
    model.eval()
    total_loss = 0.0
    correct = 0
    with torch.no_grad():
        for data, target in loader:
            data, target = data.to(DEVICE), target.to(DEVICE)
            output = model(data)
            total_loss += criterion(output, target).item() * data.size(0)
            correct += output.argmax(dim=1).eq(target).sum().item()
    avg_loss = total_loss / len(loader.dataset)
    accuracy = 100.0 * correct / len(loader.dataset)
    return avg_loss, accuracy

```

```

def run_training(model, train_loader, test_loader, optimizer, criterion, epochs, label):
    train_losses, test_losses, test_accs = [], [], []
    for epoch in range(1, epochs + 1):
        train_loss = train_one_epoch(model, train_loader, optimizer, criterion)
        test_loss, test_acc = evaluate(model, test_loader, criterion)
        train_losses.append(train_loss)
        test_losses.append(test_loss)

```

```

test_accs.append(test_acc)
print(f"[{label}] Эпоха {epoch}/{epochs}: train_loss={train_loss:.4f}, "
      f"test_loss={test_loss:.4f}, test_acc={test_acc:.2f}%")
return train_losses, test_losses, test_accs

```

```
criterion = nn.CrossEntropyLoss()
```

```

custom_cnn = SimpleCNN().to(DEVICE)
custom_optimizer = optim.Adadelta(custom_cnn.parameters(), lr=1.0)
print("Обучение CNN ")
cnn_train_losses, cnn_test_losses, cnn_test_accs = run_training(
    custom_cnn, cnn_train_loader, cnn_test_loader, custom_optimizer, criterion,
    CUSTOM_CNN_EPOCHS, "CustomCNN"
)

```

```

resnet_model = build_pretrained_resnet34().to(DEVICE)
resnet_optimizer = optim.Adadelta(
    filter(lambda p: p.requires_grad, resnet_model.parameters()), lr=1.0
)
print("\nДообучение предобученной ResNet34 (заморожены все слои кроме layer4 и fc)")
resnet_train_losses, resnet_test_losses, resnet_test_accs = run_training(
    resnet_model, resnet_train_loader, resnet_test_loader, resnet_optimizer, criterion,
    RESNET_EPOCHS, "ResNet34"
)

```

```

print(f"\nИтоговая точность кастомной CNN: {cnn_test_accs[-1]:.2f}%")
print(f"Итоговая точность предобученной ResNet34: {resnet_test_accs[-1]:.2f}%")

```

```

plt.figure(figsize=(9, 6))
plt.plot(range(1, CUSTOM_CNN_EPOCHS + 1), cnn_train_losses, label="CustomCNN train",
marker="o")
plt.plot(range(1, CUSTOM_CNN_EPOCHS + 1), cnn_test_losses, label="CustomCNN test",
marker="o")
plt.plot(range(1, RESNET_EPOCHS + 1), resnet_train_losses, label="ResNet34 train",
marker="s")
plt.plot(range(1, RESNET_EPOCHS + 1), resnet_test_losses, label="ResNet34 test", marker="s")
plt.xlabel("Эпоха")
plt.ylabel("Loss (CrossEntropyLoss)")
plt.title("Сравнение ошибки: кастомная CNN vs предобученная ResNet34 (MNIST, Adadelta)")
plt.legend()
plt.grid(True)
plt.savefig("lab2_loss_comparison.png", dpi=150, bbox_inches="tight")
plt.show()

```

```

plt.figure(figsize=(9, 6))
plt.plot(range(1, CUSTOM_CNN_EPOCHS + 1), cnn_test_accs, label="CustomCNN accuracy",
marker="o")
plt.plot(range(1, RESNET_EPOCHS + 1), resnet_test_accs, label="ResNet34 accuracy",
marker="s")
plt.xlabel("Эпоха")

```

```

plt.ylabel("Точность, %")
plt.title("Сравнение точности на тестовой выборке")
plt.legend()
plt.grid(True)
plt.savefig("lab2_accuracy_comparison.png", dpi=150, bbox_inches="tight")
plt.show()

```

```

def preprocess_for_cnn(path: str, invert: bool = True):
    image = Image.open(path).convert("L").resize((28, 28))
    arr = np.array(image).astype(np.float32) / 255.0
    if invert:
        arr = 1.0 - arr
    arr = (arr - MNIST_MEAN[0]) / MNIST_STD[0]
    tensor = torch.tensor(arr, dtype=torch.float32).unsqueeze(0).unsqueeze(0)
    return tensor, arr

```

```

def preprocess_for_resnet(path: str, invert: bool = True):
    image = Image.open(path).convert("L").resize((RESNET_IMAGE_SIZE,
RESNET_IMAGE_SIZE))
    arr = np.array(image).astype(np.float32) / 255.0
    if invert:
        arr = 1.0 - arr
    rgb = np.stack([arr, arr, arr], axis=0)
    for c in range(3):
        rgb[c] = (rgb[c] - IMAGENET_MEAN[c]) / IMAGENET_STD[c]
    tensor = torch.tensor(rgb, dtype=torch.float32).unsqueeze(0)
    return tensor, arr

```

```

def visualize_both_models(cnn_model, resnet_model, image_path: str, invert: bool = True):
    cnn_model.eval()
    resnet_model.eval()

    cnn_tensor, cnn_img = preprocess_for_cnn(image_path, invert)
    resnet_tensor, resnet_img = preprocess_for_resnet(image_path, invert)

    with torch.no_grad():
        cnn_probs = F.softmax(cnn_model(cnn_tensor.to(DEVICE)), dim=1).cpu().numpy().flatten()
        resnet_probs = F.softmax(resnet_model(resnet_tensor.to(DEVICE)),
dim=1).cpu().numpy().flatten()

    cnn_pred = int(np.argmax(cnn_probs))
    resnet_pred = int(np.argmax(resnet_probs))

    fig, axes = plt.subplots(2, 2, figsize=(10, 8))

    axes[0, 0].imshow(cnn_img, cmap="gray")
    axes[0, 0].set_title(f"Вход для CustomCNN\nПредсказание: {cnn_pred}")
    axes[0, 0].axis("off")

```

```

axes[0, 1].bar(range(10), cnn_probs)
axes[0, 1].set_title("Вероятности (CustomCNN)")
axes[0, 1].set_xticks(range(10))

axes[1, 0].imshow(resnet_img, cmap="gray")
axes[1, 0].set_title(f"Вход для ResNet34\nПредсказание: {resnet_pred}")
axes[1, 0].axis("off")

axes[1, 1].bar(range(10), resnet_probs)
axes[1, 1].set_title("Вероятности (ResNet34)")
axes[1, 1].set_xticks(range(10))

plt.tight_layout()
plt.savefig("lab2_custom_image_prediction.png", dpi=150, bbox_inches="tight")
plt.show()

print(f"\nФайл: {image_path}")
print(f"CustomCNN предсказал: {cnn_pred}, вероятности: {np.round(cnn_probs, 3)}")
print(f"ResNet34 предсказал: {resnet_pred}, вероятности: {np.round(resnet_probs, 3)}")

def visualize_from_test_set(cnn_model, resnet_model, index: int = None):
    if index is None:
        index = random.randint(0, len(cnn_test_dataset) - 1)

    cnn_image, true_label = cnn_test_dataset[index]
    resnet_image, _ = resnet_test_dataset[index]

    cnn_model.eval()
    resnet_model.eval()
    with torch.no_grad():
        cnn_probs = F.softmax(cnn_model(cnn_image.unsqueeze(0).to(DEVICE)),
dim=1).cpu().numpy().flatten()
        resnet_probs = F.softmax(resnet_model(resnet_image.unsqueeze(0).to(DEVICE)),
dim=1).cpu().numpy().flatten()

    cnn_pred = int(np.argmax(cnn_probs))
    resnet_pred = int(np.argmax(resnet_probs))

    print(f"\nТестовое изображение №{index}, истинная метка: {true_label}")
    print(f"CustomCNN предсказал: {cnn_pred}")
    print(f"ResNet34 предсказал: {resnet_pred}")

visualize_from_test_set(custom_cnn, resnet_model)

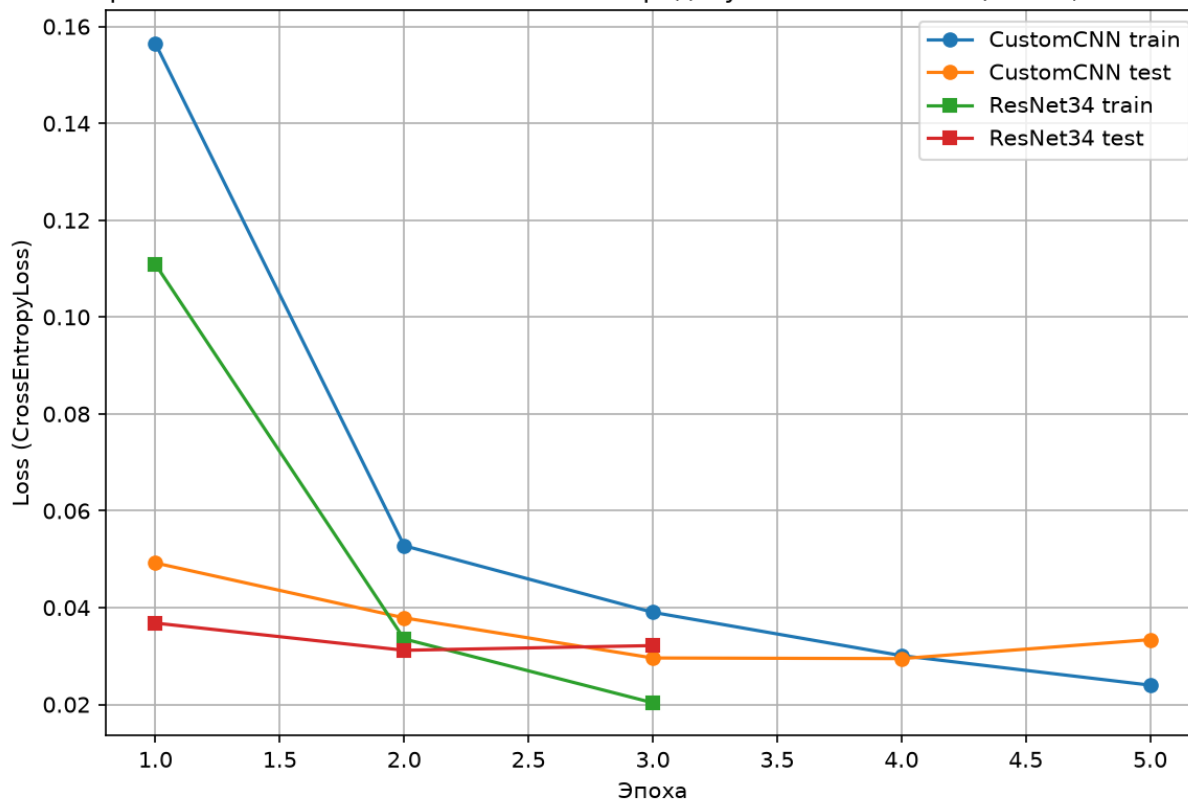
custom_image_path = "my_digit.png"
if os.path.exists(custom_image_path):
    visualize_both_models(custom_cnn, resnet_model, custom_image_path, invert=True)
else:
    print(

```

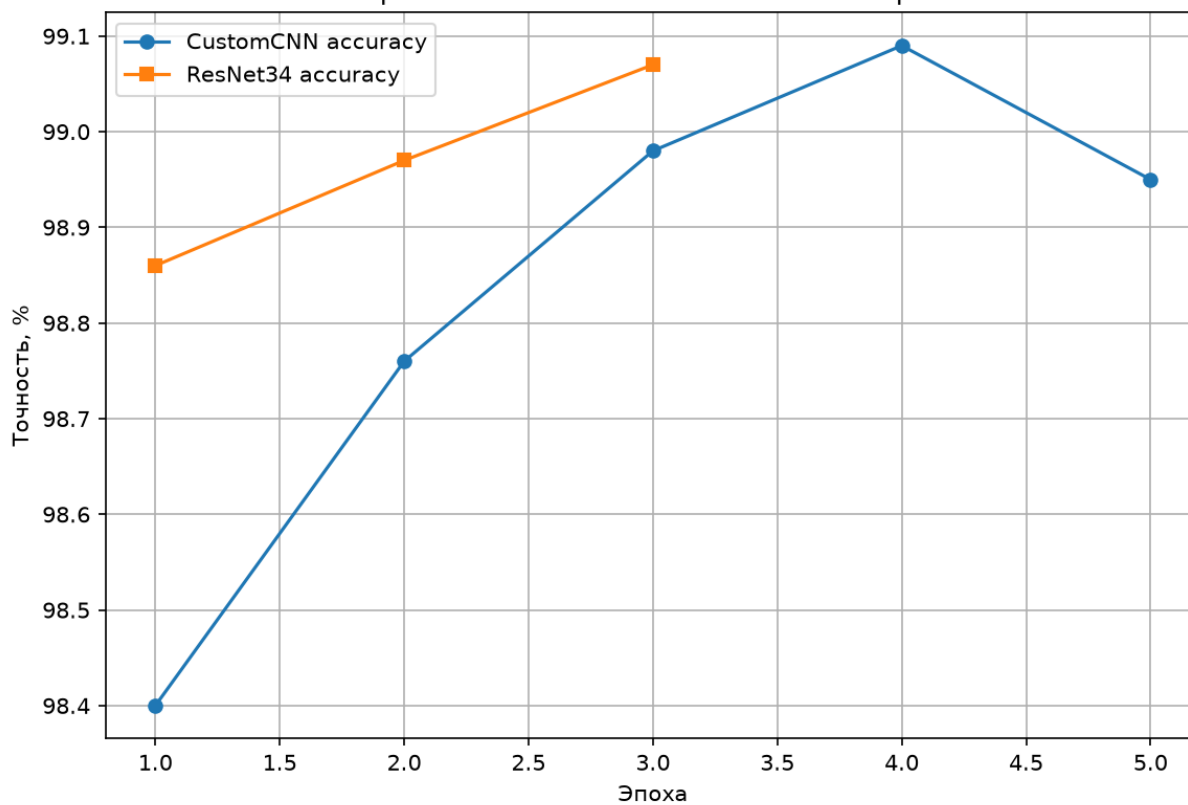
f"\nФайл {custom_image_path} не найден. Положи изображение цифры "
f"(например, скачанное из интернета или нарисованное самостоятельно) "
f"рядом со скриптом под этим именем, чтобы проверить обе модели на нём."

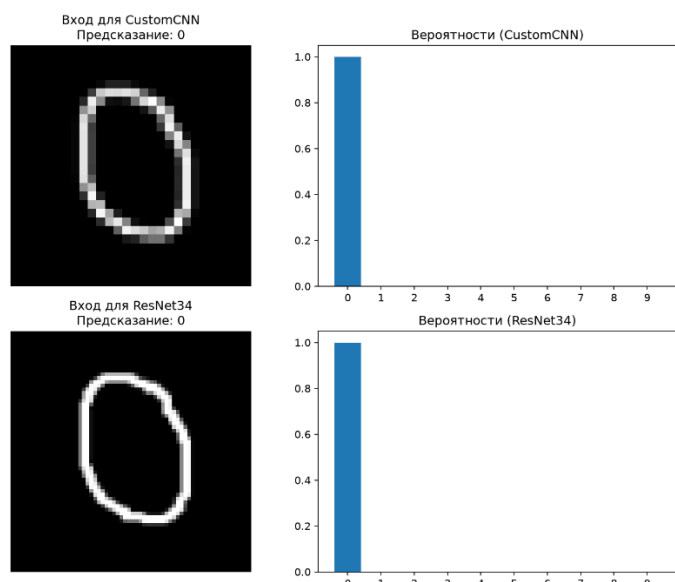
)

Сравнение ошибки: кастомная CNN vs предобученная ResNet34 (MNIST, Adadelata)



Сравнение точности на тестовой выборке





Обучение CNN

```
[CustomCNN] Эпоха 1/5: train_loss=0.1565, test_loss=0.0492, test_acc=98.40%
[CustomCNN] Эпоха 2/5: train_loss=0.0528, test_loss=0.0379, test_acc=98.76%
[CustomCNN] Эпоха 3/5: train_loss=0.0390, test_loss=0.0296, test_acc=98.98%
[CustomCNN] Эпоха 4/5: train_loss=0.0301, test_loss=0.0295, test_acc=99.09%
[CustomCNN] Эпоха 5/5: train_loss=0.0240, test_loss=0.0334, test_acc=98.95%
```

Дообучение предобученной ResNet34 (заморожены все слои кроме layer4 и fc)

```
[ResNet34] Эпоха 1/3: train_loss=0.1109, test_loss=0.0368, test_acc=98.86%
[ResNet34] Эпоха 2/3: train_loss=0.0335, test_loss=0.0312, test_acc=98.97%
[ResNet34] Эпоха 3/3: train_loss=0.0204, test_loss=0.0322, test_acc=99.07%
```

Итоговая точность кастомной CNN: 98.95%

Итоговая точность предобученной ResNet34: 99.07%

Тестовое изображение №1824, истинная метка: 2

CustomCNN предсказал: 2

ResNet34 предсказал: 2

Файл: my_digit.png

CustomCNN предсказал: 0, вероятности: [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

ResNet34 предсказал: 0, вероятности: [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

Обе модели — кастомная CNN и дообученная ResNet34 — достигли близких итоговых результатов (98.95% и 99.07% соответственно), при этом ResNet34 показала чуть более высокую и стабильную точность за меньшее число эпох (3 против 5), что говорит о пользе предобученных признаков даже при обучении лишь части слоёв. Разница в качестве при этом минимальна (~0.12) и не оправдывает существенно больший размер и вычислительную стоимость ResNet34 для такой простой задачи, как MNIST. На собственном изображении цифры «0» обе сети дали одинаковое и абсолютно уверенное предсказание (вероятность 1.0), что подтверждает хорошую обобщающую способность обеих архитектур даже на данных, отличающихся от исходной обучающей выборки.

Вывод: осуществила обучение НС, сконструированных на базе предобученных архитектур НС